

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



АВТОМАТИЗОВАНЕ
ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ

Лабораторна робота №3

“Реалізація серверної (апаратної) та клієнтської (програмної) частин гри”

Виконав:
студент гр. КІ-404
Піскурський
С. В.

Прийняв:
Федак П. Р.

Львів – 2024

Завдання:

1. Develop Server and Client.
2. Required steps.

Індивідуальне завдання:

Student number	Game	config format
15	tik-tac-toe 3x3	INI

Опис гри "Хрестики-Нолики" для Arduino та програмного клієнта

Ця програма реалізує класичну гру "Хрестики-Нолики" з трьома режимами:

1. **Людина проти людини** – двоє гравців, які по черзі роблять ходи через одну програму.
2. **Людина проти штучного інтелекту** – один гравець грає проти програми, де штучний інтелект здійснює ходи за допомогою алгоритму.
3. **Штучний інтелект проти штучного інтелекту** – два штучних інтелекти змагаються між собою без участі людини.

Гра працює за таким принципом:

- **Модель:** апаратна частина на базі Arduino є моделлю гри. Вона відповідає за перевірку правильності ходів і визначення результату (перемога, нічия або гра триває).
- **Контролер:** програма-клієнт, яка взаємодіє з Arduino через UART. Вона отримує інформацію від сервера, передає ходи і зберігає поточний стан гри.
- **View:** інтерфейс користувача, через який людина може побачити поточний стан гри та зробити свій хід. Клієнт відображає гру у текстовому вигляді на екрані.

Архітектура та взаємодія

1. **Комунікація через UART:**
 - Клієнт (програмна частина) відправляє хід на сервер (Arduino) через UART.
 - Сервер (Arduino) отримує хід, оновлює модель гри і відправляє поточний стан гри назад до клієнта.
 - Контролер на стороні клієнта обробляє ці дані, оновлюючи вигляд гри на екрані.
2. **Ігрові режими:**
 - У режимі "**Людина проти людини**" гравці по черзі роблять ходи, кожен вибираючи свою позицію на дошці.
 - У режимі "**Людина проти штучного інтелекту**" гравець робить хід, а штучний інтелект визначає свій хід.
 - У режимі "**Штучний інтелект проти штучного інтелекту**" два штучних інтелекти по черзі здійснюють ходи до визначення переможця або нічий.
3. **Перевірка результату гри:**
 - Перевірка на перемогу або нічийний результат здійснюється на сервері (Arduino). Як тільки один з гравців виграє або всі клітинки заповнені, Arduino визначає результат і передає інформацію клієнту.
4. **MVC Патерн:**
 - **Model (Модель):** апаратна частина (Arduino), яка відповідає за перевірку перемоги та управління ходами.

- **View (Представлення):** програма на клієнтській стороні, яка відображає гру користувачеві.
- **Controller (Контролер):** програмна частина на клієнті, яка обробляє взаємодію між користувачем та моделлю.

Завантаження та збереження конфігурації та стану гри

Завдяки використанню файлу конфігурації `configuration.ini`, програма на клієнтській стороні може завантажувати та зберігати налаштування гри та стан гри. Це дозволяє зберігати прогрес гри між сесіями та налаштовувати параметри гри.

Файл `configuration.ini` має такий вигляд:

```
[GameConfig]
mode=0
player_1=X
player_2=0

[GameState]
board=X034X6709
current_turn=player_1
mode=0
player_1=X
player_2=0
```

Виконання:

Ось детальний опис діаграми архітектури проекту, виконаний перед етапом розробки, що ілюструє взаємодію між компонентами клієнтської та серверної частин гри в хрестики-нулики, виконаної з використанням патерну проектування MVC:

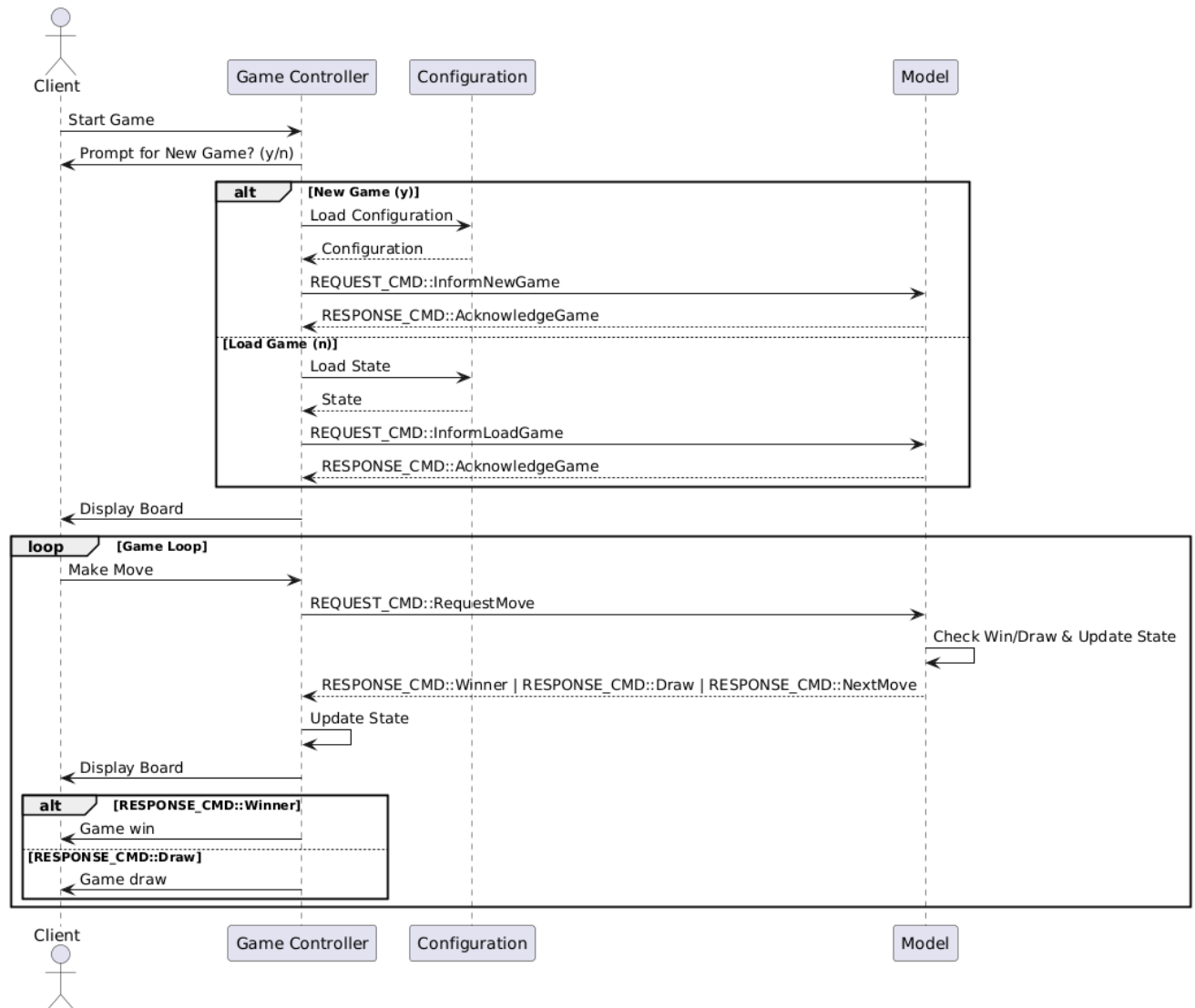


Рис.1. Архітектура гри взаємодії між сервером та клієнтом

Опис кроків діаграми

1. Start Game:

- Клієнт (SW) ініціює гру, звертаючись до Контролера (Controller).
- Контролер запитує у користувача, чи хоче він почати нову гру (Prompt for New Game?).

2. New Game (y):

- Якщо користувач вибирає нову гру (y), Контролер завантажує конфігурацію гри через компонент **Configuration (CONF)**.
- Контролер надсилає команду REQUEST_CMD::InformNewGame до **Model (HW)**, повідомляючи про початок нової гри.

- **Model (HW)** підтверджує отримання цього запиту, повертаючи RESPONSE_CMD::AcknowledgeGame.

3. Load Game (n):

- Якщо користувач вибирає завантаження попередньої гри (n), Контролер запитує стан гри з компоненту **Configuration (CONF)**.
- Контролер надсилає команду REQUEST_CMD::InformLoadGame до **Model (HW)** для завантаження попереднього стану гри.
- **Model (HW)** підтверджує отримання запиту та відправляє RESPONSE_CMD::AcknowledgeGame.

4. Display Board:

- Після завантаження конфігурації чи стану гри Контролер передає клієнту (SW) поточний вигляд ігрової дошки для відображення на екрані.

5. Game Loop:

- У циклі гри, кожен хід виглядає так:
 - Клієнт (SW) робить хід, передаючи його Контролеру.
 - Контролер передає хід на **Model (HW)**, викликаючи команду REQUEST_CMD::RequestMove.
 - **Model (HW)** перевіряє, чи є переможець або нічия, оновлює стан гри та надсилає одну з наступних відповідей:
 - RESPONSE_CMD::Winner — якщо є переможець.
 - RESPONSE_CMD::Draw — якщо гра закінчилася нічийним результатом.
 - RESPONSE_CMD::NextMove — якщо гра продовжується, і черга ходу переходить до іншого гравця.

6. Update State:

- Контролер оновлює поточний стан гри (ігрову дошку, чергу ходу).
- Клієнт отримує оновлену інформацію і відображає новий вигляд дошки на екрані.

7. Game Result:

- Якщо у відповіді від **Model (HW)** отримано RESPONSE_CMD::Winner, Контролер повідомляє клієнту про перемогу гравця.
- Якщо у відповіді отримано RESPONSE_CMD::Draw, Контролер повідомляє клієнту про нічию.

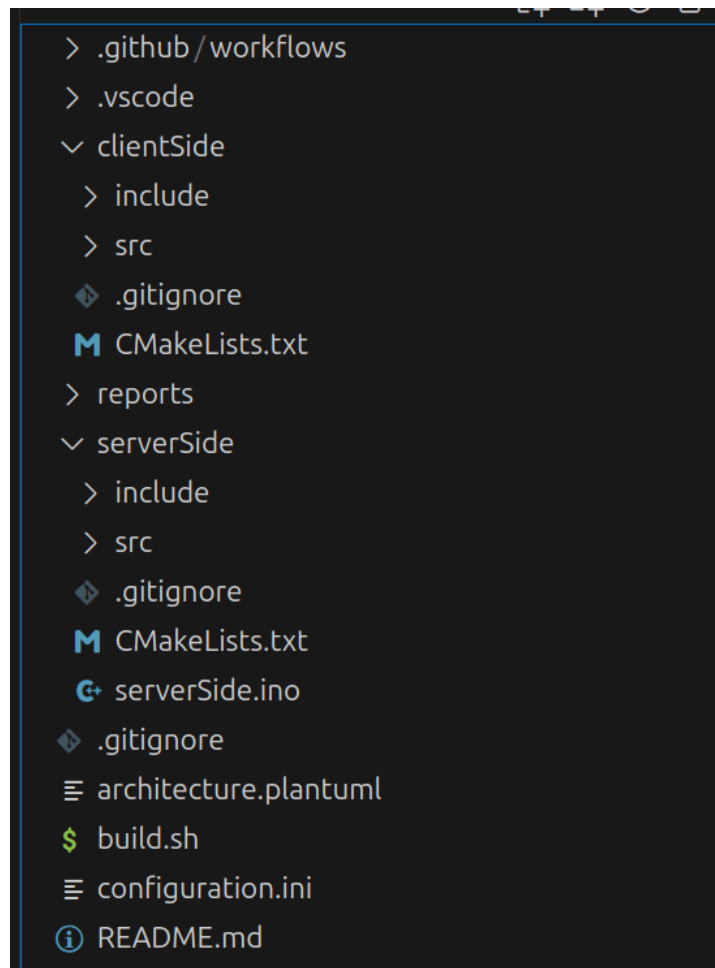


Рис.2. Оновлена структура проекту

Перевірка роботи гри в режимі Man Vs Ai (DEBUG logs enabled):

```
serhii@serhii-VirtualBox csad2425ki404piskurskyisv15 ./clientSide/build/client
Opening port '/dev/ttyUSB0'
Waiting for Arduino to be ready...
Received from Arduino: Arduino ready!

Starting Tic-Tac-Toe game...
Start a new game? (y/n): y
Game configuration loaded successfully.
Message sent: 100_0_X_0_1_123456789$
Response from Arduino: 200

Acknowledgment of game start received from hardware.

-----
1 | 2 | 3
-----
4 | 5 | 6
-----
7 | 8 | 9
-----
Player 1(X) enter your move (1-9): 5
Message sent: 102_4$
Response from Arduino: 201

-----
1 | 2 | 3
-----
4 | X | 6
-----
7 | 8 | 9
-----
AI (Player 2(O) chose position 2
Message sent: 102_1$
Response from Arduino: 201
```

```

-----
1 | 0 | 3
-----
4 | X | 6
-----
7 | 8 | 9
-----
Player 1(X) enter your move (1-9): 1
Message sent: 102_0$
Response from Arduino: 201

-----
X | 0 | 3
-----
4 | X | 6
-----
7 | 8 | 9
-----
AI (Player 2(O) chose position 8
Message sent: 102_7$
Response from Arduino: 201

-----
X | 0 | 3
-----
4 | X | 6
-----
7 | 0 | 9
-----
Player 1(X) enter your move (1-9): 9
Message sent: 102_8$
Response from Arduino: 202

-----
X | 0 | 3
-----
4 | X | 6
-----
7 | 0 | X
-----
WE HAVE A WINNER - X !!!

```

Розробив утиліту build.sh, яка дозволяє зручно будувати та керувати проектами клієнта та сервера. Опис параметрів та функцій скрипту:

1. Основні параметри:

- **client:** будує тільки проект клієнта.
- **server:** будує тільки проект сервера. Якщо вказано, додатково виконується завантаження прошивки на сервер.
- **clean:** очищає всі створені каталоги збудованих файлів та документацію для клієнта та сервера.
- **-h або --help:** виводить інформацію про використання скрипту.

2. Загальні дії:

- Якщо не вказано конкретну опцію, скрипт будує як клієнт, так і сервер за замовчуванням.
- Скрипт може приймати додатковий параметр для вказівки серійного порту для сервера (якщо вибрано побудову сервера).

```
serhii@serhii-VirtualBox csad2425ki404piskurskyisv15 bash build.sh -h
Usage: build.sh [OPTION] [SERIAL_PORT]
Build and manage client and server projects.

Options:
  client      Build only the client project.
  server      Build only the server project (includes upload target).
  clean       Remove build directories for both client and server.
  -h, --help  Display this help and exit.

If no option is specified, both client and server will be built.
To specify a custom serial port for the server, pass it as an additional argument.
```

Команди користування утилітою також розписав в README.MD.

Висновок:

Виконуючи дану лабораторну роботу я розробив логіку гри для клієнта і сервера та налаштував комунікацію між ними використовуючи клієнт-серверну архітектуру спілкування через UART. Також використав CMake для налаштування і компіляції клієнтського та серверного проєктів. Розробив скрипт *build.sh* для зручного будування програм. Після ручного тестування розробленого функціоналу, додав зміни, закомітив їх, створив пул реквест та добавив викладача в рев'юери.